

Arreglos Paginados

Proyecto #1 — Algoritmos y Estructuras de Datos II (CE 2103)

Josue Adrian Perez Perez
Escuela de Ingeniería en Computadores
Instituto Tecnológico de Costa Rica
I Semestre 2026
j.perez.1@estudiantec.cr

Resumen—Este proyecto implementa una clase `PagedArray` la cual simula la lógica de paginación de memoria, la cual permite ordenar archivos binarios utilizando cinco algoritmos de ordenamiento sin modificar la lógica interna, además se analizan las consecuencias de modificar los parámetros de la paginación sobre el rendimiento de cada algoritmo, esto se hace al medir los tiempos de ejecución, los page faults y page hits, esto revela que el tamaño de página y la cantidad de páginas cargadas son factores determinantes para el rendimiento utilizando paginación.

Index Terms—paginación, algoritmos de ordenamiento, page fault, page hit, administración de memoria

I. CONTEXTO DEL PROBLEMA

Actualmente, la administración de memoria representa uno de los desafíos al diseñar sistemas y aplicaciones, la memoria de un sistema, que se conoce como RAM, es un recurso finito que nunca es suficiente para tener todos los datos que requiere un programa, especialmente cuando se trabaja con datos de gran escala que se encuentran en megabytes o incluso en gigabytes.

En base a esto, actualmente se han desarrollado técnicas que permiten extender la capacidad efectiva de la memoria, esto gracias al almacenamiento al utilizarlo como una extensión de la RAM al dividir los datos en bloques denominados como páginas y manteniendo en RAM únicamente aquellas páginas que se necesiten en un momento dado, lo que no se necesite permanece almacenado en el disco hasta que se necesite.

Este proyecto implementa el concepto de paginación mediante la implementación de una clase denominada `PagedArray`, la cual encapsula la lógica de paginación y se comporta como un arreglo convencional de C++, lo cual permite que diferentes algoritmos operen sobre esta clase, esto, además de mostrar de manera práctica el funcionamiento de estos sistemas de manejo de memoria, también permite observar el impacto que tienen los page faults y los page hits en el rendimiento de diferentes algoritmos de ordenamiento.

El proyecto comprende dos programas, un generador de archivos binarios de enteros aleatorios y un ordenador que implementa cinco algoritmos de ordenamiento sobre la clase `PagedArray`, los cuales poseen diferentes formas de acceder a memoria, por lo cual, permiten ser comparados en base a tiempo de ejecución, cantidad de page faults y cantidad de page hits.

II. DETALLES DE IMPLEMENTACIÓN

II-A. `PagedArray`

La clase `PagedArray` es la parte principal del proyecto, ya que contiene toda la lógica de paginación que utilizan los algoritmos de ordenamiento, esta clase expone una interfaz mínima, la cual consiste en un constructor, un destructor y una sobrecarga del operador de indexación, manteniendo toda la lógica interna encapsulada en la sección privada de la clase.

De forma interna, la clase administra un arreglo de punteros de tipo `Page`, cada uno posee un arreglo dinámico de enteros de 32 bits, el cual representa el contenido de una página en memoria, también posee un número de página y un indicador de modificación denominado `isDirty`, el cual indica si una página es alterada y debe de escribirse en el disco antes de ser reemplazada, además, la clase también posee contadores de page faults y page hits para las estadísticas finales.

El constructor recibe parámetros como la ruta del archivo binario, el tamaño de página y la cantidad máxima de páginas que se pueden tener en memoria, luego procede a abrir el archivo en modo de lectura y escritura binaria, calcula el tamaño total del arreglo y aloja dinámicamente los arreglos internos inicializando cada espacio de página con un negativo uno, el cual indica una ausencia de página. El destructor garantiza la eliminación de los recursos alocados, escribiendo en el disco cualquier página que haya sido modificada y que aun permanezca en memoria.

La otra parte importante de la clase es la sobrecarga del operador, ya que este es el que implementa el manejo de las páginas siguiendo un flujo definido de pasos: dado un índice cualquiera, calcula el número de página mediante división entera entre el índice y el tamaño de página, y el desplazamiento dentro de esa página mediante el operador módulo, luego se busca si la página que se necesita ya se encuentra en la memoria, en el mejor caso, se aumenta el contador de page hits y se retorna una referencia directa al elemento, en el peor caso, se incrementa page fault y se procede a cargar la página desde disco aplicando el algoritmo de reemplazo LRU.

II-B. Algoritmo de Reemplazo LRU

El algoritmo de reemplazo seleccionado para este proyecto es LRU, cuyo funcionamiento consiste en tener un contador de accesos en `PAGE`, cuando se requiere de cambiar una página se busca la página que tiene la menor cantidad de accesos y se intercambia por la página a la que se necesita acceder.

II-C. Generator

El programa generador recibe en la linea de comandos dos argumentos, un tamaño predefinido de los cuales hay 3 opciones, SMALL, MEDIUM o LARGE, los cuales corresponden a 128MB, 256MB y 512MB respectivamente (32MB, 64MB y 128MB para la defensa del proyecto), y una ruta de salida donde se guardara el archivo generado, el programa valida estas entradas y general el archivo escribiendo enteros aleatorios mediante un buffer de 1,048,576 enteros utilizando `fwrite`, el contenido del archivo es completamente binario y no legible ya que cada entero es serializado directamente en sus 4 bytes sin utilizar ningún tipo de codificación o separador.

II-D. Sorter

El ordenador recibe mediante la linea de comandos cinco argumentos, la ruta del archivo de entrada, la ruta del archivo de salida, el nombre del algoritmo de ordenamiento, el tamaño de pagina y la cantidad de paginas, luego de hacer una validación de los argumentos, el programa produce una copia del archivo de entrada, el cual es el archivo de salida, luego crea un objeto `PagedArray` con los parametros necesarios, luego se ejecuta el algoritmo seleccionado midiendo el tiempo de ejecución utilizando `chrono`, luego se genera un archivo de texto con los enteros ordenados separados por comas, y finalmente se imprime un resumen con el algoritmo que se utilizo, el tiempo de ejecución en segundos, la cantidad de page hits y page faults.

II-E. Algoritmos de Ordenamiento

Se realizo la implementacion de cinco algoritmos de ordenamiento, donde solo se modifiko la firma de estos para aceptar un objeto `PagedArray`, manteniendo la lógica interna intacta, los algoritmos que se implementaron fueron Bubble Sort con su complejidad $O(n^2)$, Insertion Sort con complejidad $O(n^2)$, Merge Sort con $O(n \log n)$, Quick Sort con $O(n \log n)$ en el caso promedio, y Quick Sort con selección pivote Mediana de Tres con la misma complejidad pero con un comportamiento mas estable, dado que los algoritmos cuadráticos son inviables para archivos de gran tamaño, el programa restringe Bubble Sort e Insertion Sort a un tamaño denominado TEST el cual es de 512KB, mientras que los demás algoritmos si soportan los tamaños predefinidos.

III. DETALLES DE EXPERIMENTACIÓN

III-A. Metodología General

Las pruebas tuvieron como objetivo caracterizar el comportamiento de cada algoritmo bajo distintas configuraciones, buscando relaciones entre los parámetros de `pageSize` y `pageCount` y métricas de rendimiento como el tiempo de ejecución, la cantidad de page faults y la cantidad de page hits. En cada combinación de algoritmo y tamaño se utilizaron cuatro configuraciones distintas:

- `pageSize 100, pageCount 5`
- `pageSize 100, pageCount 50`
- `pageSize 1,000,000, pageCount 5`
- `pageSize 1,000,000, pageCount 50`

III-B. Configuraciones de Paginación

Estas cuatro configuraciones presentan cuatro extremos, la combinación de `pageSize` bajo con `pageCount` bajo representa el escenario de mayor restricción de memoria, con páginas pequeñas y pocos espacios disponibles, lo que genera una alta frecuencia de page faults. La combinación de `pageSize` alto con `pageCount` bajo representa el escenario de mayor eficiencia esperada, con páginas grandes que reducen la frecuencia de carga desde disco.

IV. RESULTADOS DE EXPERIMENTACIÓN

IV-A. Merge Sort

Merge Sort resulto ser el algoritmo mas eficiente del conjunto para los archivos MEDIUM y LARGE, completando el ordenamiento en todas las configuracion con tiempos menores a los demas algoritmos. (Ver apéndice B, Cuadro I)

Los resultados confirmaron que tener un `pageSize` alto con un `pageCount` bajo produjo los mejores tiempos, alcanzando 39.45 segundos en SMALL, 71.35 segundos en MEDIUM y 92.24 segundos en LARGE, algo que se resalta es que aumentar `pageCount` de 5 a 50 con el mismo `pageSize` incremento en gran medida el tiempo de ejecución, siendo la diferencia de aproximadamente 64 segundos en SMALL, 62 segundos en MEDIUM y 186 segundos en LARGE.

IV-B. Quick Sort

Quick sort mostró que este mismo es un algoritmo versátil, ya que completo el ordenamiento en todas las configuraciones, pero con tiempos mayores que Merge Sort en las configuraciones mas favorables. (Ver apéndice B, Cuadro II)

Quick Sort mostró los mismos resultados que Merge Sort respecto a `pageSize` y `pagecount`, con `pagesize` alto y `pageCount` bajo teniendo los mejores resultados, pero, a diferencia de Merge Sort, Quick Sort fue mas rápido en SMALL alcanzando 24.94 segundos contra los 39.45 segundos de Merge Sort, aunque fue mas lento en LARGE.

IV-C. Quick Sort Mediana de Tres

Quick Sort con selección de pivote Mediana de Tres mostró un comportamiento similar al Quick Sort estándar, con tiempos ligeramente mejores en algunas configuraciones pero sin diferencias determinantes. (Ver apéndice B, Cuadro III)

Un resultado contraintuitivo en Quick Sort M3 es que, a pesar de generar consistentemente menos page faults que Quick Sort estándar, sus tiempos de ejecución fueron mayores en la mayoría de las configuraciones.

IV-D. Bubble Sort e Insertion Sort

Bubble Sort e Insertion Sort fueron evaluados únicamente sobre el archivo de prueba TEST debido a su complejidad cuadrática, que los hace inviables para archivos de mayor tamaño bajo restricciones de memoria paginada. (Ver apéndice B, Cuadro IV y V)

Los resultados de Bubble Sort e Insertion Sort revelan dos patrones importantes. Primero, Insertion Sort fue consistentemente más rápido que Bubble Sort en todas las configuraciones, siendo la diferencia más notable en la configuración de

pageSize 100 y pageCount 5 donde Insertion Sort tardó 169.51 segundos frente a los 399.88 de Bubble Sort. Segundo, ambos algoritmos muestran el mismo patrón negativo de pageCount alto que los algoritmos eficientes, con tiempos significativamente mayores al aumentar pageCount de 5 a 50 independientemente del pageSize.

IV-E. Comparación con Baseline

Se puede apreciar que el overhead general utilizando el PagedArray ronda entre 3.2 y 2.4, esto significa que el utilizar PagedArray tiene implicaciones negativas en los tiempos de ejecución, lo cual se puede deber a los procesos de lectura y escritura de las paginas o el algoritmo de reemplazo.

IV-F. Comparación General

Estos resultados mostraron que hay mejores opciones para el rendimiento en archivos de mediano y gran tamaño, siendo la mejor opción Merge Sort en las configuraciones más favorables, luego esta Quick Sort y finalmente Quick Sort M3, el cual mostro tiempos mayores a pesar de que produjera menos page faults, otro punto interesante es que tener un pageCount alto termino siendo perjudicial para todos los algoritmos, por otro lado, un pageSize alto permitio que se obtuvieran bastantes buenos tiempos, lo uqe confirma que el tamaño de pagina es el parametro con mayor impacto

Los resultados de la experimentación permiten establecer una jerarquía clara de rendimiento entre los algoritmos implementados para archivos de mediano y gran tamaño, con Merge Sort liderando en las configuraciones más favorables seguido de cerca por Quick Sort, mientras que Quick Sort M3 mostró tiempos mayores a pesar de generar menos page faults. El hallazgo transversal más significativo de toda la experimentación fue que un pageCount alto resultó perjudicial para todos los algoritmos sin excepción, mientras que un pageSize alto fue condición necesaria para obtener tiempos competitivos, confirmando que el tamaño de página es el parámetro de paginación con mayor impacto sobre el rendimiento de cualquier algoritmo de ordenamiento bajo restricciones de memoria paginada.

V. CONCLUSIONES

El desarrollo de este proyecto permitió comprender de forma práctica y profunda el concepto de paginación de memoria, demostrando que la administración eficiente de los recursos del sistema no es únicamente responsabilidad del sistema operativo sino que puede y debe ser considerada activamente por el programador al diseñar soluciones de software que operan sobre conjuntos de datos de gran escala. La implementación de la clase PagedArray demostró que es posible encapsular completamente la lógica de paginación detrás de una interfaz familiar y transparente, permitiendo que algoritmos estándar operen sobre ella sin ninguna modificación a su lógica interna.

Uno de los hallazgos más significativos de la experimentación fue la confirmación de que la complejidad algorítmica teórica no es suficiente por sí sola para predecir el rendimiento real de un algoritmo cuando opera bajo restricciones de

memoria paginada. Esto quedó ilustrado de forma particular en el caso de Quick Sort M3, que a pesar de generar consistentemente menos page faults que Quick Sort estándar demostró tiempos de ejecución mayores, evidenciando que el número de page faults no es el único factor determinante del rendimiento y que el patrón de acceso interno del algoritmo puede interactuar de formas complejas con el algoritmo de reemplazo de páginas utilizado.

La experimentación con distintas configuraciones de pageSize y pageCount reveló que el tamaño de página es el parámetro verdaderamente crítico para la viabilidad y el rendimiento de cualquier algoritmo de ordenamiento bajo restricciones de memoria paginada, siendo un pageSize extremadamente bajo de 100 altamente perjudicial para todos los algoritmos. Adicionalmente, el efecto negativo de un pageCount alto resultó ser un hallazgo contraintuitivo pero consistente, sugiriendo que el algoritmo de reemplazo LRU interactúa negativamente con los patrones de acceso de los algoritmos evaluados cuando hay muchas páginas simultáneas en memoria.

Desde una perspectiva más amplia, este proyecto demostró que el estudio de algoritmos y estructuras de datos no puede desvincularse del estudio de la arquitectura y administración de los sistemas sobre los que estos operan, ya que decisiones de diseño aparentemente menores como la elección del algoritmo de reemplazo de páginas o el tamaño de las páginas pueden tener un impacto mayor sobre el rendimiento real que la propia complejidad teórica del algoritmo de ordenamiento utilizado.

REPOSITORIO DE GITHUB

El código fuente completo del proyecto, incluyendo instrucciones de compilación y ejecución, se encuentra disponible en el siguiente repositorio: https://github.com/JosueAPerez/Proyecto_1_CE2103

APÉNDICE

A. Prompts Utilizados para Agentes de IA

A lo largo del proyecto se solicitó la ayuda del agente de IA Claude para que sirviera como consultor al momento de surgir dudas puntuales sobre la realización del proyecto. Las consultas realizadas se limitaron a aspectos didácticos como explicación de conceptos, sintaxis de funciones de C++ y toma de decisiones de diseño, sin solicitar en ningún momento que el agente produjera código que resolviera directamente los requerimientos del proyecto. A continuación se adjuntan los prompts específicos utilizados:

- “¿Cómo funciona lo de los comandos en consola, porque aún no comprendo cómo funciona lo de pasar argumentos y cómo ejecutar los códigos?”
- “¿Qué me recomiendas hacer para generar los números aleatorios? ¿Hay un tipo de dato que sea un entero de 32 bits?”
- “¿Cómo me recomiendas escribir en el archivo, ya que hacerlo de uno en uno sería demasiado lento?”

- “¿Cómo me recomiendas hacer la clase PagedArray?
¿Sería buena idea hacer clases con el nombre Page?”
- “Explicame cómo funciona lo de sobrecargar el operador[].”
- “¿Cuál sería la mejor configuración de pageSize y pageCount para hacer las pruebas?”
- “¿Qué algoritmos de ordenamiento hay más eficientes que los básicos para intentar mejorar los tiempos?”
- “Explicame cómo funciona LRU y cómo implementarlo.”
- “¿Qué parámetros serían mejor para hacer las pruebas?”
- “Revisemos el código del PagedArray y del sorter para verificar que no haya errores o malas prácticas.”

B. Cuadros

Cuadro I
RESULTADOS DE MERGE SORT

Tamaño	Page Size	Page Count	Tiempo (s)	Page Faults	Page Hits
SMALL	100	5	67.53	11,995,713	1,665,725,887
SMALL	100	50	163.84	9,075,427	1,668,646,173
SMALL	1,000,000	5	39.45	258	1,677,721,342
SMALL	1,000,000	50	103.44	34	1,677,721,566
MEDIUM	100	5	183.25	25,333,589	3,464,327,339
MEDIUM	100	50	416.12	19,493,021	3,470,167,907
MEDIUM	1,000,000	5	71.35	670	3,489,660,258
MEDIUM	1,000,000	50	133.07	204	3,489,660,724
LARGE	100	5	176.52	53,351,554	7,194,405,758
LARGE	100	50	408.82	41,670,414	7,206,086,898
LARGE	1,000,000	5	92.24	1,609	7,247,755,703
LARGE	1,000,000	50	278.46	677	7,247,756,635

Cuadro II
RESULTADOS DE QUICK SORT

Tamaño	Page Size	Page Count	Tiempo (s)	Page Faults	Page Hits
SMALL	100	5	42.27	10,993,004	2,112,002,033
SMALL	100	50	113.52	8,523,527	2,114,471,510
SMALL	1,000,000	5	24.94	215	2,122,994,822
SMALL	1,000,000	50	74.41	34	2,122,995,003
MEDIUM	100	5	90.97	24,112,043	4,535,504,320
MEDIUM	100	50	245.71	19,151,937	4,540,464,426
MEDIUM	1,000,000	5	52.93	584	4,559,615,779
MEDIUM	1,000,000	50	204.37	130	4,559,616,233
LARGE	100	5	194.37	52,556,372	9,674,806,575
LARGE	100	50	527.33	42,640,104	9,684,722,843
LARGE	1,000,000	5	111.45	1,508	9,727,361,439
LARGE	1,000,000	50	409.96	565	9,727,362,382

Cuadro III
RESULTADOS DE QUICK SORT MEDIANA DE TRES

Tamaño	Page Size	Page Count	Tiempo (s)	Page Faults	Page Hits
SMALL	100	5	41.05	9,978,390	2,054,880,189
SMALL	100	50	110.06	7,759,562	2,057,099,017
SMALL	1,000,000	5	24.30	209	2,064,858,370
SMALL	1,000,000	50	72.45	34	2,064,858,545
MEDIUM	100	5	146.67	20,855,820	4,244,996,863
MEDIUM	100	50	395.30	16,429,928	4,249,422,755
MEDIUM	1,000,000	5	82.99	509	4,265,852,174
MEDIUM	1,000,000	50	323.51	129	4,265,852,554
LARGE	100	5	304.86	44,074,395	8,751,809,710
LARGE	100	50	818.48	35,224,183	8,760,659,922
LARGE	1,000,000	5	176.35	1,253	8,795,882,852
LARGE	1,000,000	50	610.08	426	8,795,883,679

Cuadro IV
RESULTADOS DE BUBBLE SORT E INSERTION SORT (ARCHIVO TEST)

Algoritmo	Page Size	Page Count	Tiempo (s)	Page Faults	Page Hits
Bubble	100	5	399.88	85,963,397	25,669,626,051
Bubble	100	50	1323.16	85,837,442	25,669,752,006
Bubble	1,000	5	285.28	8,640,509	25,746,948,939
Bubble	1,000	50	1063.37	7,380,554	25,748,208,894
Insertion	100	5	169.51	43,002,442	12,821,167,769
Insertion	100	50	619.58	42,524,641	12,821,645,570
Insertion	1,000	5	120.92	4,359,729	12,859,810,482
Insertion	1,000	50	499.81	2,519,523	12,861,650,688

Cuadro V
RESULTADOS DE BASELINE CONTRA PAGEDARRAY (ARCHIVO SMALL)

Algoritmo	Código	Tiempo
Quick	Baseline	10.1494
Quickm3	Baseline	10.4982
Merge	Baseline	11.454
Quick	PagedArray	32.4032
Quickm3	PagedArray	32.3532
Merge	PagedArray	28.4675